Amelia Singh

05 April 2026

Music Management With ID3v1 Tags in Java

This project builds a framework for music library management and metadata editing in a direct and extensible manner, allowing for Java interfacing with the existing ID3v1 specification which is used standardly by the majority of existing MP3 players. The frontend created here is an interactive user-friendly library browser and queue manager, with some QOL (quality of life) improvements like automatic playlist sorting and title autocompletion, as was the scope of the project. The file interfacing on the backend could however be easily extended to solve other problems like batch metadata editing, and automating file organization.

The ID3v1 Specification

The ID3 specification for MP3 metadata was released one year after the release of the MP3 format in 1995, at the time of which, there was no standard for storing information. The first version of which, ID3v1, suggested using the last 128 bytes of the file to store ISO-8859-1 encoded strings of fixed lengths. The benefits of this are simplicity, explicit offsets from the end of the file, as well as not disrupting playback on MP3 players without built-in support, as it's stored after the audio data. The downsides are limitations on types of data stored, number of fields, and the length of each item.

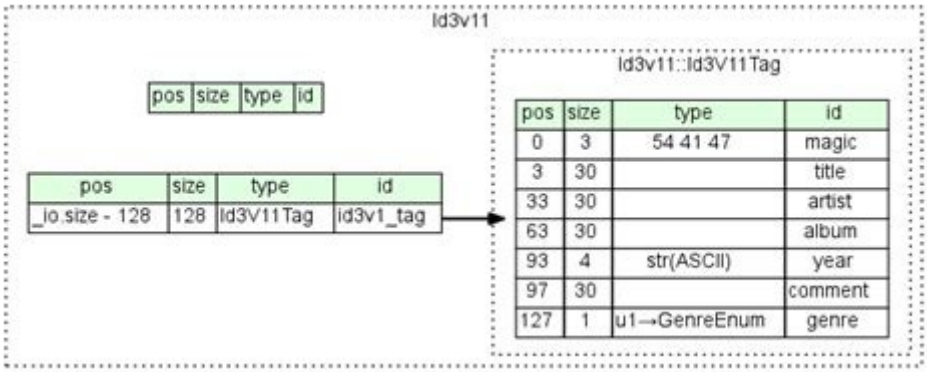

Figure 1: Diagram of the ID3v1 format

The second version, ID3v2, solves these problems, but due to the added complexity of its implementation it was a less ideal option for this project. It does so by using variable length fields at the beginning of the file, allowing variable data types and lengths, and a variable number of fields. It's also more optimal for streaming as the metadata will be loaded before the audio data. As data being streamed is downloaded in chunks rather than one full file at a time. However, ID3v1 provides the necessary information in a simpler manner and the order of data is of little relevance as we are not dealing with streaming, and the majority of modern MP3 tagging software will add both types of tags together.

Project Structure

The project is broken up into three main classes; Song, Songs (extended for Albums and Playlists to handle naming separately), and Interface, as well as classes for the implementation of a few data types; a linked list based stack, a linked list based queue, a singly linked list, and our generic node.

Our Song class takes in a Path and reads metadata from the file into variables which it stores. It uses a Hashmap to map genre codes to human readable genre names in an efficient manner. The Songs class stores an ArrayList of Song items, and automatically sorts them by title field using a recursive implementation of quick sort, this was chosen to reduce space complexity while still prioritizing speed. There are two subclasses of Songs, Album and Playlist, which handle naming from the file metadata, and Path name respectively. The Interface class makes a linked list queue to store queued Songs and a linked list stack to store history. It then provides functions to play, pause, skip, show metadata, manage directories, etc., which are exposed to the user via a shell-like interface.

Figure 2: Screen capture of the user interface

The main data structure I chose not to use was a Binary Search Tree, simply because loading files into a BST would be an O(n) operation, and it would reduce searching from O(n) to O(log n), but for such an infrequent operation in such a small data set, it wasn't worth the additional complexity it would add to the other functions that rely on File I/O.

Further Development

There are a few main ways this program could be reasonably adapted into a releasable product. Firstly, the interface could be extended to be more user friendly and accessible with consideration to the average person, and functionality could be added for handling audio output. Secondly, the backend could be extended to fit a few other use cases, namely metadata editing, and file organization. However, both of these would require similar adaptation with respect to the user interface.

In summary, this program provides a reasonable foundation for the development of Java-based tools that interface with ID3-tagged audio files; namely music players, and library organization tools, and demonstrates one such use case. In order to cater to mass-market needs, it would be reasonable to consider the development of a Graphical User Interface, with special consideration for accessibility and ease of use, as well as support for generic consumer audio equipment, and the newest version of the ID3 specification, to ensure the widest file support.